

1.

```
temperaturas = []

temperaturas.append(820)
temperaturas.append(835)
temperaturas.append(848)
temperaturas.append(791)
temperaturas.append(812)
temperaturas.append(856)

print(f"Leituras: {temperaturas}")
print(f"Máximo: {max(temperaturas)} °C")
print(f"Mínimo: {min(temperaturas)} °C")
```

2.

```
leituras_originais = [11.9, 12.1, 12.0, 13.5, 11.7, 12.3, 10.5, 12.0]

amostras = []
for valor in leituras_originais:
    amostras.append(valor)

media = sum(amostras) / len(amostras)
print(f"Média: {media:.2f} V")

for v in amostras:
    if v < 10.8 or v > 13.2:
        print(f"ALERTA: {v} V fora da faixa!")
```

3.

```
tamanhos_entrada = [512, 1024, 1500, 1518, 256, 1500, 2048, 64, 1500, 1024]

buffer = []

for tamanho in tamanhos_entrada:
    buffer.append(tamanho)

total_pacotes = len(buffer)
total_bytes = sum(buffer)

pacotes_jumbo = sum(1 for p in buffer if p > 1500)
porcentagem_jumbo = (pacotes_jumbo / total_pacotes) * 100
print(f"Pacotes recebidos : {total_pacotes}")
print(f"Total de bytes      : {total_bytes}")
print(f"Pacotes jumbo        : {pacotes_jumbo} ({porcentagem_jumbo:.1f}%)")
```

4.

```
vibracao = [0.12, 0.34, -999.0, 0.28, 0.91, -999.0, 1.42, 0.67]
```

```
while -999.0 in vibracao:  
    vibracao.remove(-999.0)
```

```
print(f"Leituras válidas: {vibracao}")  
print(f"Quantidade : {len(vibracao)}")
```

5.

```
cargas = [15.0, 22.5, 8.0, 40.0, 12.5, 30.0]
```

```
potencia_antes = sum(cargas)
```

```
cargas.remove(8.0) # Desligada por sobrecarga  
cargas.remove(40.0) # Equipamento em manutenção
```

```
potencia_agora = sum(cargas)  
reducao = potencia_antes - potencia_agora
```

```
print(f"Cargas ativas : {cargas}")  
print(f"Potência antes: {potencia_antes:.1f} kW")  
print(f"Potência agora: {potencia_agora:.1f} kW")  
print(f"Redução : {reducao:.1f} kW")
```

6.

```
canais = [850, 900, 950, 1800, 2100, 2600]
```

```
interferencias = {  
    850: -65,  
    900: -80,  
    950: -72,  
    1800: -55,  
    2100: -90,  
    2600: -68  
}
```

```
}  
limiar = -70
```

```
canais_remove = []  
canais_ativos = []
```

```
for canal in canais:
```

```
    nivel = interferencias[canal]
```

```
    if nivel > limiar: # Lembre-se: em valores negativos, -65 > -70
```

```
        canais_remove.append(canal)
```

```
        print(f"Canal {canal} MHz desativado (interferência: {nivel} dBm)")
```

```
    else:
        canais_ativos.append(canal)

print(f"Canais ativos: {canais_ativos}")
```

```
7.
params_originais = [120.0, 150.0, 95.0, 200.0, 110.0]

backup = params_originais.copy()

params_originais[0] = 130.0

print(f"Original (modificada): {params_originais}")
print(f"Backup (preservado): {backup}")
```

```
8.
nominal = [47.0, 100.0, 220.0, 33.0, 68.0]

simulacao = nominal.copy()

indice = simulacao.index(100.0)
simulacao[indice] = 0.0

req_nominal = sum(nominal)
req_falha = sum(simulacao)
variacao = req_nominal - req_falha

print(f"Nominal      : {nominal}")
print(f"Simulação    : {simulacao}")
print(f"Req nominal   : {req_nominal} Ω")
print(f"Req falha     : {req_falha} Ω")
print(f"Variação      : {variacao} Ω")
```

9.

```
tabela_atual = ['10.0.0.0/8', '172.16.0.0/12', '192.168.1.0/24', '203.0.113.0/24',  
'198.51.100.0/24']
```

```
rota_removida = '203.0.113.0/24'
```

```
rotas_inseridas = ['10.10.0.0/16', '192.168.100.0/24']
```

```
tabela_antiga = tabela_atual.copy()
```

```
if rota_removida in tabela_atual:
```

```
    tabela_atual.remove(rota_removida)
```

```
tabela_atual.extend(rotas_inseridas)
```

```
rotas_adicionadas = [r for r in tabela_atual if r not in tabela_antiga]
```

```
rotas_removidas = [r for r in tabela_antiga if r not in tabela_atual]
```

```
print(f"Rotas adicionadas: {rotas_adicionadas}")
```

```
print(f"Rotas removidas : {rotas_removidas}")
```

10.

```
sensor_A = [3.2, 3.4, 3.1, 3.5]
```

```
sensor_B = [4.1, 4.0, 4.3, 3.9]
```

```
sensor_C = [2.8, 2.9, 3.0, 2.7]
```

```
leituras_totais = []
```

```
leituras_totais.extend(sensor_A)
```

```
leituras_totais.extend(sensor_B)
```

```
leituras_totais.extend(sensor_C)
```

```
total_leituras = len(leituras_totais)
```

```
pressao_media = sum(leituras_totais) / total_leituras
```

```
pressao_maxima = max(leituras_totais)
```

```
pressao_minima = min(leituras_totais)
```

```
print(f"Total de leituras: {total_leituras}")
```

```
print(f"Pressão média : {pressao_media:.2f} bar")
```

```
print(f"Pressão máxima : {pressao_maxima} bar")
```

```
print(f"Pressão mínima : {pressao_minima} bar")
```

11.

```
medidor_1 = [48.2, 51.0, 49.7, 47.5, 52.3]
```

```
medidor_2 = [50.1, 46.8, 53.0, 49.0, 55.2]
```

```
medicoes_total = medidor_1.copy()
```

```
medicoes_total.extend(medidor_2)
```

```
medicoes_total.sort()
```

```
media = sum(medicoes_total) / len(medicoes_total)
```

```
tolerancia = media * 0.15
```

```
limite_inferior = media - tolerancia
```

```
limite_superior = media + tolerancia
```

```
fora_da_faixa = [m for m in medicoes_total if m < limite_inferior or m > limite_superior]
```

```
print(f"Medições ordenadas: {medicoes_total}")
```

```
print(f"Média: {media:.2f} Ω | Tolerância: ±{tolerancia:.2f} Ω")
```

```
if not fora_da_faixa:
```

```
    print("(nenhum valor fora da faixa neste conjunto)")
```

```
else:
```

```
    print(f"Valores fora da faixa: {fora_da_faixa}")
```

12.

```
antena_norte = [-85, -90, -105, -78, -92, -110, -88]
```

```
antena_sul = [-95, -88, -102, -79, -85, -91, -97]
```

```
antena_leste = [-80, -75, -108, -93, -86, -103, -82]
```

```
limiar = -100
```

```
todos_logs = []
```

```
todos_logs.extend(antena_norte)
```

```
todos_logs.extend(antena_sul)
```

```
todos_logs.extend(antena_leste)
```

```
total_amostras = len(todos_logs)
```

```
cobertura_adequada = [leitura for leitura in todos_logs if leitura >= limiar]
```

```
em_cobertura = len(cobertura_adequada)
```

```
taxa_cobertura = (em_cobertura / total_amostras) * 100
```

```
pior_sinal = min(todos_logs)
```

```
indice_pior = todos_logs.index(pior_sinal)
```

```
media = sum(todos_logs) / total_amostras
```

```

print(f"Total de amostras : {total_amostras}")
print(f"Em cobertura (>= -100): {em_cobertura}")
print(f"Taxa de cobertura : {taxa_cobertura:.1f}%")
print(f"Pior sinal : {pior_sinal} dBm (índice {indice_pior})")
print(f"Média : {media:.1f} dBm")

```

13.

```

medicoes = [50.02, 49.97, 50.06, 49.94, 50.00, 50.08, 49.99, 50.01, 49.92, 50.04]
nominal = 50.00
tolerancia = 0.05

```

```

aprovadas = []
reprovadas = []

```

```

for medida in medicoes:
    # Verifica se a peça está dentro da faixa de tolerância
    if abs(medida - nominal) <= tolerancia:
        aprovadas.append(medida)
    else:
        reprovadas.append(medida)

```

```

indice_aprovacao = (len(aprovadas) / len(medicoes)) * 100

```

```

print(f"Aprovadas ({len(aprovadas)}): {aprovadas}")
print(f"Reprovadas ({len(reprovadas)}): {reprovadas}")
print(f"Índice de aprovação: {indice_aprovacao:.0f}%")

```

14.

```

cenario_atual = [15.0, 22.5, 8.0, 40.0, 12.5, 30.0]
novas_cargas = [55.0, 18.0, 75.0]

```

```

cenario_futuro = cenario_atual.copy()
cenario_futuro.extend(novas_cargas)

```

```

def exibir_resumo(nome, lista):
    total = sum(lista)
    media = total / len(lista)
    maior = max(lista)
    print(f"--- {nome} ---")
    print(f"Total : {total:.1f} kW | Média : {media:.1f} kW | Maior : {maior:.1f} kW")

```

```

exibir_resumo("Cenário atual", cenario_atual)

```

```

exibir_resumo("Cenário futuro", cenario_futuro)
15.
import math

canal_I = [0.82, 0.79, 0.85, 5.20, 0.81, 0.78, 0.83]
canal_Q = [0.41, 0.39, 0.43, 0.40, -4.80, 0.42, 0.38]

sinal_bruto = []
sinal_bruto.extend(canal_I)
sinal_bruto.extend(canal_Q)

backup_sinal = sinal_bruto.copy()

n = len(sinal_bruto)
mu = sum(sinal_bruto) / n
soma_quadrados = sum((x - mu) ** 2 for x in sinal_bruto)
sigma = math.sqrt(soma_quadrados / n)

ruidos = []
limite_superior = mu + 3 * sigma
limite_inferior = mu - 3 * sigma

for amostra in sinal_bruto:
    if amostra > limite_superior or amostra < limite_inferior:
        ruidos.append(amostra)

for r in ruidos:
    sinal_bruto.remove(r)

print(f"Sinal bruto ({len(backup_sinal)} amostras): {backup_sinal}")
print(f"Ruídos removidos: {ruidos}")
print(f"Sinal limpo ({len(sinal_bruto)} amostras): {sinal_bruto}")

```